# The Complexity Status of Graph Isomorphism

## From Factorial Brute Force to Quasi-Polynomial Breakthrough

Presented by: *Theory Presentation Team*

March 27, 2026

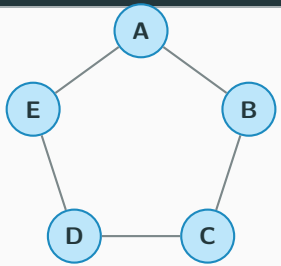
CSE, BUET

# Outline

1. Introduction — What is Graph Isomorphism?
2. Complexity — Where Does GI Live?
3. Why Computers Struggle — The Matrix Problem
4. Babai's Breakthrough — 2015
5. The Weisfeiler-Lehman Test — Solving GI in Practice
6. Algorithms — A Progression
7. Special Cases — When GI Is Easy
8. Real-World Applications

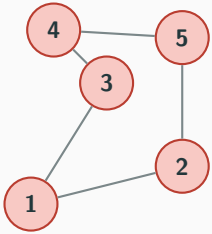
# Introduction — What is Graph Isomorphism?

# Are These the Same Graph?




Graph $G$

?




Graph $H$

**★ Human Intuition Fails Here**

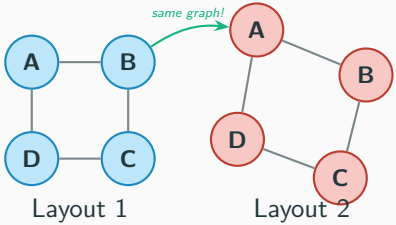
They *look* different — but are they structurally identical?
This is the **Graph Isomorphism Problem**.

# The Word Isomorphism

## Etymology

- **iso** (Greek) *equal*
- **morphe** (Greek) *shape*

⇒ **"Same shape"**




## The Necklace Analogy

Tangle, stretch, or rearrange a necklace — the *connections* never change. The **structure** is what matters, not the picture.

# Formal Definition

**Definition — Graph Isomorphism**

Two graphs $G = (V_G, E_G)$ and $H = (V_H, E_H)$ are **isomorphic** ($G \cong H$) if there exists a bijection:

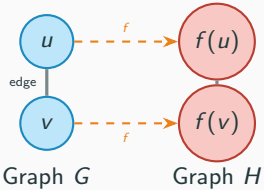
$$f : V_G \longrightarrow V_H$$

such that for all $u, v \in V_G$:

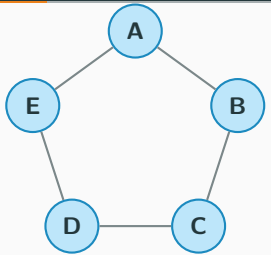
$$(u, v) \in E_G \iff (f(u), f(v)) \in E_H$$

**In plain English:**

- If $A$ and $B$ are friends in $G$, their twins $f(A)$, $f(B)$ must be friends in $H$.
- No friendship is added or removed.
- It is a *structure-preserving* renaming.

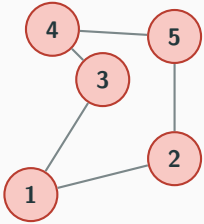


# Revealing the Mapping




$G$

| $G \to H$ |
|---|
| A $\to$ 1 |
| B $\to$ 2 |
| C $\to$ 5 |
| D $\to$ 4 |
| E $\to$ 3 |




$H$

$G \cong H$ — **They are the same graph!**

The mapping $f$ is the proof.

# Why is This a Problem?

**Brute-Force Approach:**

- Try *every possible* mapping $f$
- For $n$ nodes: $n!$ possibilities

**Scale of $n!$**

| Nodes | $n!$ |
|---|---|
| 5 | 120 |
| 10 | 3,628,800 |
| 20 | $\approx 2.4 \times 10^{18}$ |
| 100 | $9.33 \times 10^{157}$ |

$\otimes$ Atoms in the observable universe:

$$\approx 10^{80}$$

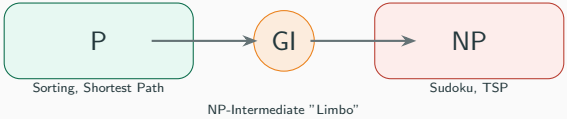
$100! = 9.33 \times 10^{157}$

**Vastly exceeds** the number of atoms!

$\Rightarrow$ We need a **clever algorithm**.

# Complexity — Where Does GI Live?

# The "Map" of Computational Complexity



# Why is GI Not Known to be in P?

### ? Why Not Easy (P)?

- No polynomial algorithm found that works for *every* graph.
- Highly **symmetric** graphs defeat most shortcuts.
- After decades of effort — still open!

### ? Why Not Hard (NP-Complete)?

- NP-Complete problems have *random* structure.
- GI has rich **mathematical structure** (Group Theory).
- It's more like a Rubik's Cube than a pile of tangled yarn!

### ⊞ The Rubik's Cube Analogy

**NP-Complete:** Tangled yarn — no structure to exploit.

**GI:** A Rubik's Cube — looks messy, but strict mathematical rules allow clever "twists."

# Evidence: GI Cannot Be NP-Complete

**▷ Schöning's Theorem (1988) Schöning (1988)**

If **Graph Isomorphism** were NP-Complete,
then the **Polynomial Hierarchy** would collapse.

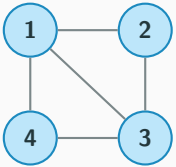
**What is the Polynomial Hierarchy?**

- A tower of complexity levels above P and NP.
- Each level is "strictly harder" than the one below.
- A *collapse* means many "hard" problems become "easy" — widely believed to be **false**.

$\Rightarrow$ GI is **almost certainly not NP-Complete**.
It lives in the mysterious **NP-Intermediate** zone.

# Why Computers Struggle — The Matrix Problem

# How a Computer "Sees" a Graph




Graph $G$

**Adjacency Matrix** of $G$:

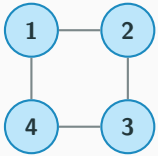
$$A_G = \begin{pmatrix} 0 & \mathbf{1} & \mathbf{1} & \mathbf{1} \\ \mathbf{1} & 0 & \mathbf{1} & 0 \\ \mathbf{1} & \mathbf{1} & 0 & \mathbf{1} \\ \mathbf{1} & 0 & \mathbf{1} & 0 \end{pmatrix}$$

- Entry $[i, j] = 1$ iff edge $(i, j)$ exists.
- Node **labels** determine the matrix layout.

# The Labelling Nightmare

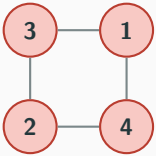
**Key insight:** The same graph gives *different* matrices for different labellings!

**Labelling 1**




$$\begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

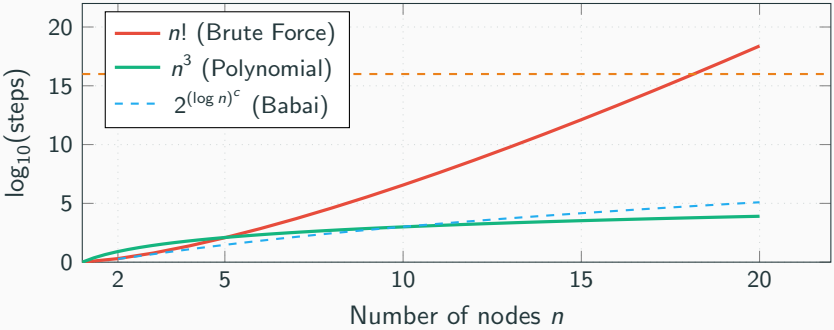
**Labelling 2 (same graph!)**




$$\begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

The computer must find: can we **permute rows & columns** of $A_G$ to get $A_H$?
For $n$ nodes $\Rightarrow$ $n!$ row/column permutations to check.

# The Factorial Explosion



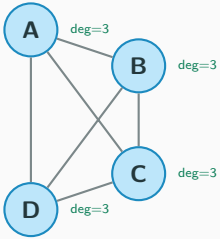

*For $n = 100$: brute force needs $\sim 10^{157}$ steps — utterly infeasible.*

# The "No Corners" Problem — Regular Graphs

**Regular Graph**

Every node has the **same degree** (same number of neighbours).

- In a jigsaw puzzle ⇒ look for *corner pieces*.
- In GI with regular graphs ⇒ **all nodes look identical**.
- No unique starting point!
- The computer gets completely lost.




$K_4$: every node has degree 3

# Babai's Breakthrough — 2015

# November 2015 — A Bombshell

**★ The Announcement**

**László Babai** (University of Chicago) announced a

**Quasi-Polynomial**

time algorithm for Graph Isomorphism Babai (2015).

- Previous best: Luks (1983) Luks (1983) — sub-exponential.
- Babai improved it by an *exponential factor*.
- Considered one of the greatest TCS results of the 21st century.

**László Babai**
Univ. of Chicago
*Computer Scientist*

# What is Quasi-Polynomial Time?

**Babai's Algorithm Runs in Time:**

$$2^{O((\log n)^c)} \quad \text{for some constant } c$$

**Exponential (Old)**

$2^n$

For $n = 100$:

$\approx 10^{30}$ steps

**Quasi-Poly (Babai)**

$2^{(\log n)^c}$

For $n = 100$:

$\approx 10^7$ steps

**Polynomial (Goal)**

$n^k$

For $n = 100$:

$\approx 10^6$ steps

**Exponential $\gg$ Quasi-Polynomial $\gg$ Polynomial**

Babai's result is the *closest* GI has ever been to the "Easy" bucket.

# Explore More: Visualize GI Complexity

**Explore More**

For an interactive visualization of Graph Isomorphism complexity (including Babai's breakthrough), visit:
https://graph-isomorphism-complexity.netlify.app/

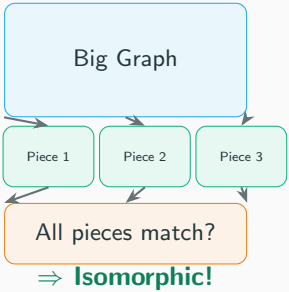
# Babai's Strategy: Divide and Conquer

**Three Key Ideas:**

1. **Symmetry Breaking**
   Hard graphs are too symmetrical.
   Find "weak" spots to break symmetry.

2. **Local Landmarks (Splitter)**
   Identify unique sub-structures within the graph.

3. **Divide & Conquer**
   Split graph into smaller pieces at landmarks.
   If all pieces match $\Rightarrow$ whole graphs match.




Uses deep results from **Group Theory**.

# The Drama Behind the Proof

**△ The Gap (Jan 2017)**

Harald Helfgott found a *gap* (error) in one of Babai's key lemmas Helfgott (2017).

The community held its breath...

**✓ The Fix (Feb 2017)**

Babai returned to his office, worked for weeks, and **fixed the proof**.

By early 2017: community consensus — *the proof is correct*.

# What Babai's Result Means

**▲ Impact**

- **Algorithmically:** Vastly faster for large graphs.
- **Theoretically:** GI is "almost" polynomial — the closest it has ever been to P.
- **Structurally:** Confirms GI has deep mathematical structure exploitable by algorithms.

*"Babai's result didn't just give us a faster tool; it proved that Graph Isomorphism is fundamentally* **not that hard**.*"*

**Open Question (Still!)**

Is GI in **P**?
No one knows. It remains one of the great open problems in TCS.

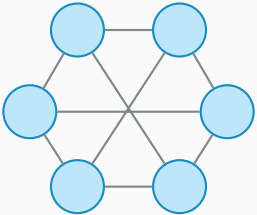
# The Weisfeiler-Lehman Test — Solving GI in Practice

# Real-World Solution: The Colour Refinement Trick

**Weisfeiler-Lehman (WL) Test Weisfeiler and Lehman (1968)**

Instead of checking structure directly, we **colour** nodes based on their neighbourhood.

If two graphs produce different colour palettes $\Rightarrow$ **not isomorphic**.

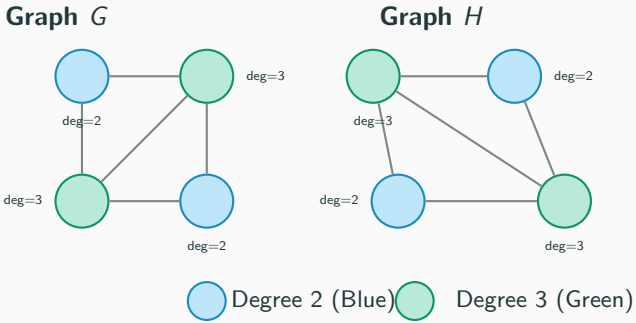


Petersen graph Petersen (1891)

**Two Phases:**

1. **Initial Colouring** — by node degree.
2. **Refinement** — update colour based on neighbours' colours.

# WL Step 1: Initial Colouring by Degree




Both graphs have **2 blue** (degree-2) and **2 green** (degree-3) nodes.
$\Rightarrow$ Initial colouring does *not* distinguish them. **Refine further!**

# WL Step 2: Refinement by Neighbourhood Colours

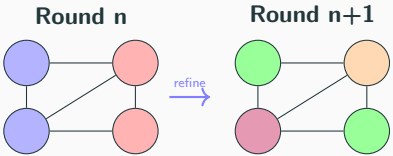
**Rule:** Update each node's colour based on *its own colour + sorted colours of neighbours*.

**Example Update**

Node **Blue** with neighbours
{**Red**, **Red**} ⇒ New color: **Purple**

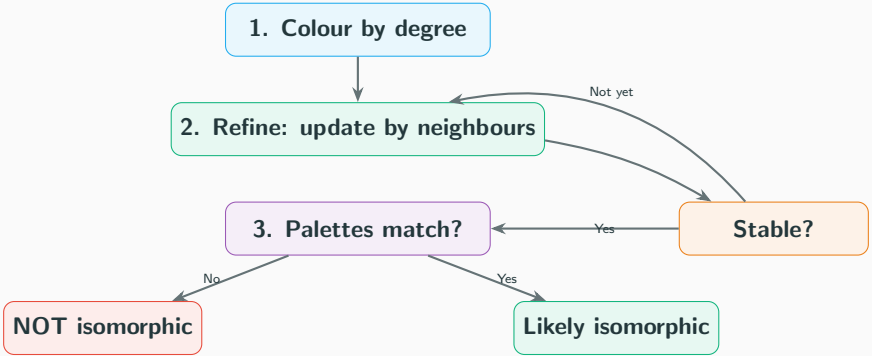
Node **Blue** with neighbours
{**Red**, **Blue**} ⇒ New color: **Green**

Repeat until colours **stabilise**[cite: 403].




*Nodes change shade because their neighbors look different!*

# WL Algorithm — Full Flow



# WL Test: Power and Weakness

## (+) Strengths

- Runs in **polynomial time** $O(n \log n)$.
- Works for almost all graphs encountered in practice.
- Used in **Graph Neural Networks** (GNNs) in AI!

## (-) Weakness

- Fails on **Cai-Fürer-Immerman (CFI) graphs** Cai et al. (1989).
- Highly regular graphs with no distinguishing local structure.
- WL says "same" but they are *different*!

## CFI Graph — The WL Adversary

WL cannot tell these apart!

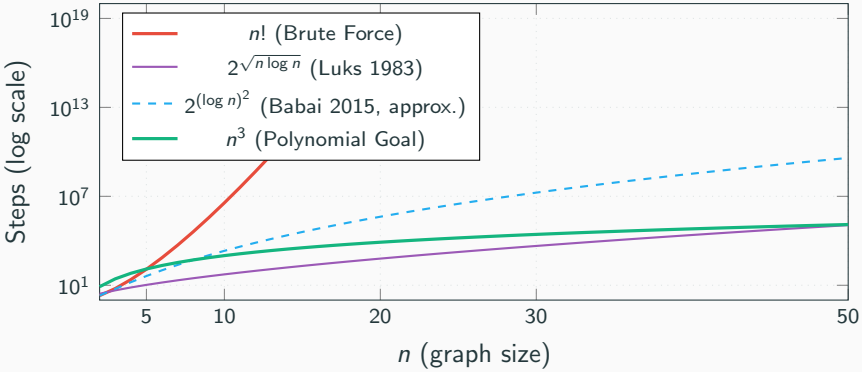
# Algorithms — A Progression

# Evolution of GI Algorithms

| Year | Author | Complexity | Key Idea |
|---|---|---|---|
| 1970s | Various | $O(n!)$ | Brute force / backtracking |
| 1980 | Luks | $2^{O(\sqrt{n \log n})}$ | Group theory, bounded degree |
| 1983 | Luks | Sub-exponential | Divide & conquer on groups |
| 1992 | Babai, Luks | $e^{O(\sqrt{n \log n})}$ | Canonical form |
| **2015** | **Babai** | $\mathbf{2^{O((\log n)^c)}}$ | **Group theory + local cert.** |

Each step is a *massive* improvement. The 2015 jump was the most dramatic in 30 years.
References: Luks (1980), Luks (1983), Babai and Luks (1992), Babai (2015)

# Growth Rate Comparison



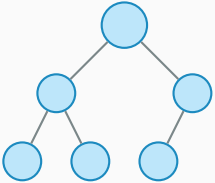
# Special Cases — When GI Is Easy

# When is GI Provably Easy?

**✓ Polynomial Cases**

- **Trees** — $O(n \log n)$
- **Planar graphs** — $O(n)$ Hopcroft and Wong (1974)
- **Bounded degree** — polynomial Luks (1980)
- **Interval graphs**
- **Bounded genus graphs**

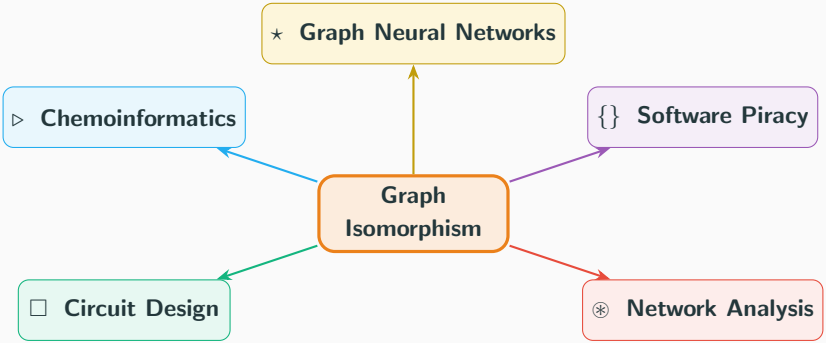
**Tree (Easy!)**



The more **structure** a graph family has, the easier GI becomes.
Random, highly-connected graphs are the hardest cases.

# Real-World Applications

# Why Does This Matter?

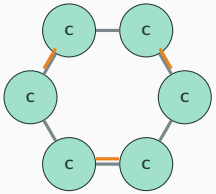


# Application 1: Chemoinformatics

**▷ The Problem**

Are two molecular structures the same compound?

Atoms = nodes. Chemical bonds = edges.



Benzene ring as a graph

**Why it matters:**

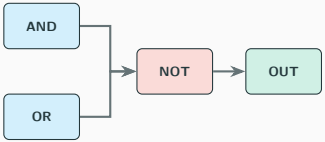
- Drug discovery: is a new compound the same as a known drug?
- Toxicology: detecting toxic isomers.
- Patent checking for novel compounds.

# Application 2: Circuit Design Verification

**The Problem**

Does the physical chip layout match the digital blueprint?

Logic gates = nodes. Wires = edges.




**Why it matters:**

- Modern chips have *billions* of transistors.
- Even one wrong connection $\Rightarrow$ chip failure.
- GI algorithms enable automated verification.

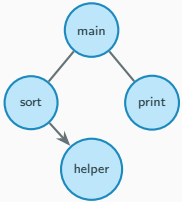
# Application 3: Software Piracy Detection

## {} The Problem

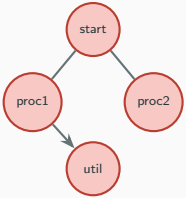
Is this code a stolen copy, even if variable names were changed?

Functions = nodes. Call relationships = edges.

**Original Code (Call Graph):**




**Stolen Code (Renamed):**



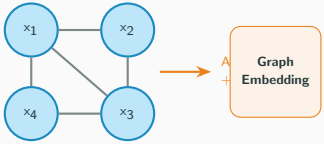

GI detects: same call structure ⇒ likely **copied code**!

# Application 4: Graph Neural Networks (AI)

**⋆ GNNs and WL**

Modern AI uses graphs to represent: molecules, social networks, knowledge bases.

- GNNs work by *aggregating* node features from neighbours.
- This is **exactly** the WL colour refinement algorithm!
- The **expressiveness** of a GNN is bounded by the WL test.
- Key result: standard GNNs cannot distinguish non-isomorphic graphs that fool WL.




WL power

= GNN power.

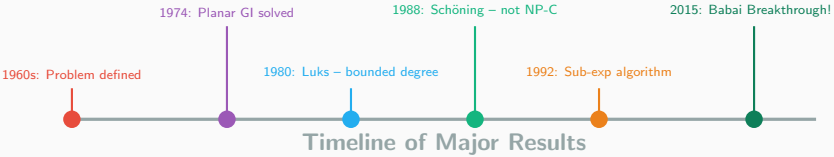
# Open Problems and Summary

# What Remains Unknown?

1. **Is GI in P?**
   The million-dollar question. We have quasi-polynomial; polynomial is still open.
2. **Can Babai's algorithm be improved to polynomial?**
   Would require fundamentally new group-theoretic machinery.
3. **Does $P \neq NP$?**
   If $P = NP$, then GI $\in$ P trivially.
4. **Is there a graph canonical form computable in polynomial time?**
   A canonical form would solve GI; its complexity remains open.

Graph Isomorphism sits at the **frontier** of complexity theory.
It has driven some of the deepest results in algorithms and group theory.

# The Full Journey of GI



# Summary — The Big Picture

### What is GI?

Determining if two graphs share the same structure via a bijection $f : V_G \to V_H$.

### Where does it live?

NP-Intermediate — harder than P, almost certainly easier than NP-Complete.

### Best algorithm?

Babai (2015): $2^{O((\log n)^c)}$ — quasi-polynomial.

### Practical solution?

Weisfeiler-Lehman colour refinement — polynomial, works on almost all real graphs.

### Open question?

Is GI $\in$ P? Nobody knows. Still an active frontier.

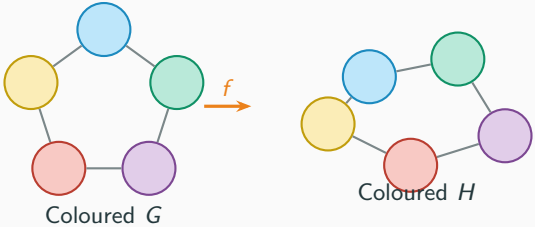
### Real-world uses?

Chemistry, circuits, code, AI/GNNs, networks.

# The Key Takeaway

**"In theory, Graph Isomorphism is a monster.**

**In practice, we tame the monster with colour refinement."**



# Thank You!

**Questions?**

# References


Babai, L. (2015). Graph isomorphism in quasipolynomial time. *arXiv preprint arXiv:1512.03547*. Breakthrough quasi-polynomial time algorithm: $2^{O((\log n)^c)}$; announced November 2015.

Babai, L. and Luks, E. M. (1992). Canonical labeling of graphs. In *Proceedings of the 24th Annual ACM Symposium on Theory of Computing*, pages 171–183. Algorithm achieving $2^{O(\sqrt{n \log n})}$ time complexity.

Cai, J.-Y., Fürer, M., and Immerman, N. (1989). An optimal lower bound on the number of variables for graph identifications. *Journal of Computer and System Sciences*, 39(2):267–290. Introduces CFI graphs that defeat the Weisfeiler-Lehman test.

Helfgott, H. A. (2017). Isomorphismes de graphes en temps quasi-polynomial [graph isomorphisms in quasipolynomial time]. *arXiv preprint arXiv:1701.04372*. Identifies and helps correct a gap in Babai's original proof.

# References ii


Hopcroft, J. E. and Wong, J. K. (1974). Linear time algorithm for isomorphism of planar graphs. In *Proceedings of the Sixth Annual ACM Symposium on Theory of Computing*, pages 172–184.

Luks, E. M. (1980). Isomorphism of graphs of bounded valence can be tested in polynomial time. In *Proceedings of the 21st Annual Symposium on Foundations of Computer Science*, pages 42–49.

Luks, E. M. (1983). Polynomial-time algorithms for permutation groups. In *Proceedings of the 24th Annual Symposium on Foundations of Computer Science*, pages 36–41. Presents sub-exponential algorithm for graph isomorphism.

Petersen, J. (1891). Die theorie der regulären graphs. In *Acta Mathematica*, volume 15, pages 193–220. Introduces the Petersen graph, a classic example of a 3-regular graph.

Schöning, U. (1988). Graph isomorphism is in the low hierarchy. *Journal of Computer and System Sciences*, 37(3):312–323. Proves GI cannot be NP-Complete unless the Polynomial Hierarchy collapses.

# References iii

Weisfeiler, B. M. and Lehman, A. A. (1968). A reduction of a graph to a canonical form and an algebra arising during this reduction. *Nauchno-Technicheskaya Informatika*, 2:12–16. Original Russian work; foundational for the WL test.